

Mlabs QMS

Target System Architecture

Complete architecture for the QMS platform — UI apps, API gateway, the Flowable workflow engine, platform services, the future backend split into domain microservices, and the data layer.

Recommended backend decomposition

Today the platform runs as a **single backend application** hosting all eleven QMS modules on one Flowable engine and one PostgreSQL schema. For the future split, the recommendation is **not** one service per module (that would turn the many synchronous cross-module triggers into fragile network calls). Instead, group the eleven modules into **six domain microservices** aligned to bounded contexts, supported by **five platform services**. That is **11 independently deployable services** in total, plus a message broker and a database per service.

The six domain services (the direct answer to “how many, and their names”):

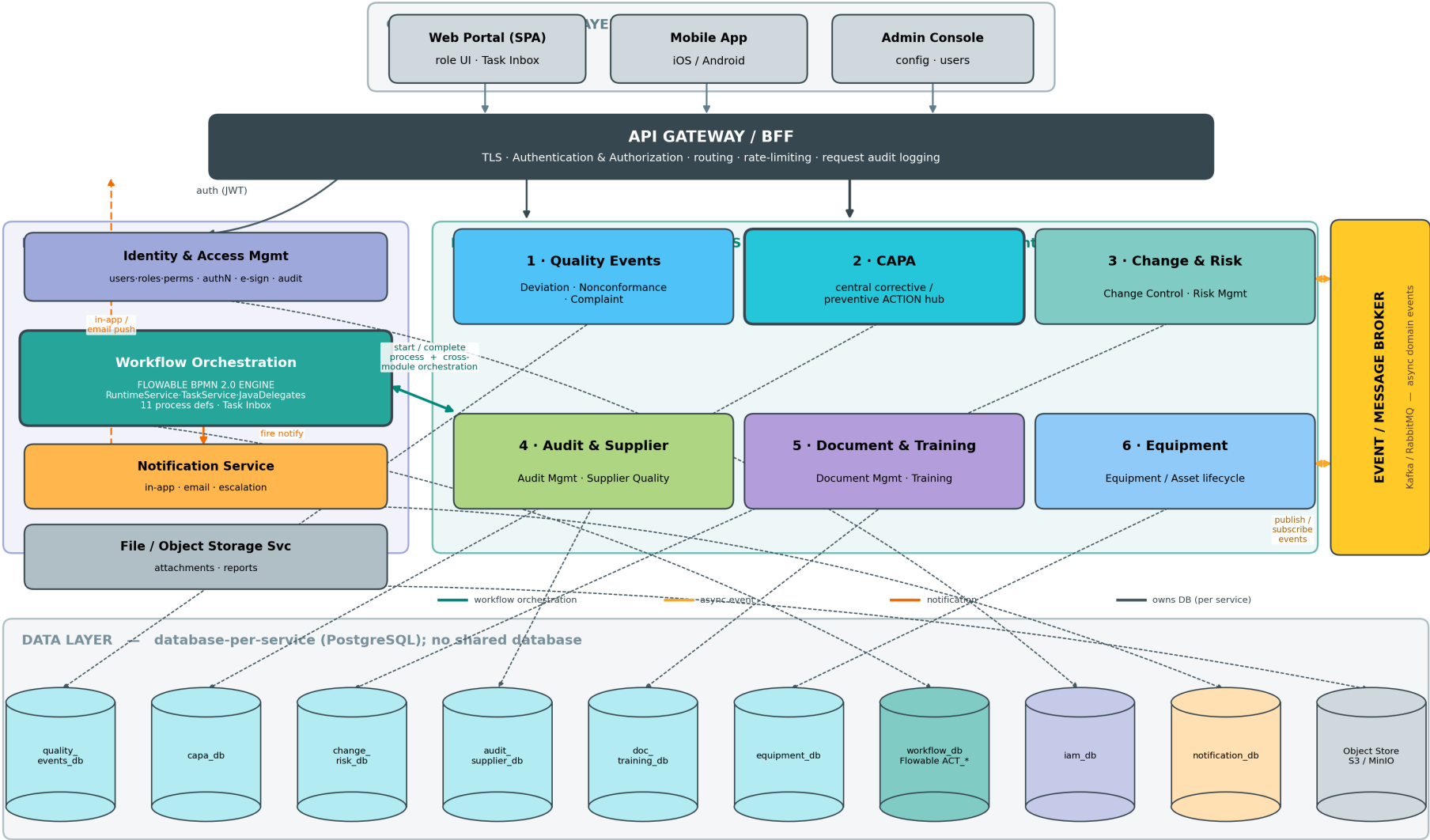
#	Domain service (deployable name)	QMS modules it owns	Why grouped together
1	Quality Events Service qms-quality-events	Deviation, Nonconformance, Complaint	Three ‘something went wrong’ event lifecycles that share intake/investigation patterns and frequently cross-link.
2	CAPA Service qms-capa	CAPA	Universal corrective/preventive hub with very high fan-in from every other module — isolated so it can scale and evolve independently.
3	Change & Risk Service qms-change-risk	Change Control, Risk Management	Risk and Change trigger each other bidirectionally and share impact-assessment logic.
4	Audit & Supplier Service qms-audit-supplier	Audit, Supplier Quality	Supplier qualification schedules and consumes supplier audits — tightly coupled.
5	Document & Training Service qms-document-training	Document Mgmt, Training	An effective/revised document directly drives training assignments; near-inseparable.
6	Equipment Service qms-equipment	Equipment Management	Largely self-contained asset lifecycle (qualification, calibration, maintenance).

Plus five platform / supporting services: **API Gateway/BFF** (qms-gateway), **Identity & Access Management** (qms-iam) which also owns e-signatures and the audit trail, **Workflow Orchestration Service** (qms-workflow) which hosts the Flowable engine and all 11 BPMN process definitions, **Notification Service** (qms-notification), and **File/Object Storage Service** (qms-storage).

Target Architecture Diagram

Mlabs QMS — Target System Architecture

Future microservices target: 6 domain services + Flowable workflow engine + platform services + database-per-service



Clients → API Gateway → platform services (left, incl. Flowable workflow engine) + six domain microservices (right grid) → event broker → database-per-service. Green = workflow orchestration; amber = async events; orange = notifications; dashed grey = each service owns its own database.

Service catalog

Every service is independently deployable, owns its data, and exposes a REST API behind the gateway.

Service	Type	Responsibility	Datastore
Web Portal / Mobile / Admin	Client	Role-based UIs and the unified Task Inbox; mobile (iOS/Android) for field tasks; admin for configuration.	–
API Gateway / BFF	Platform	Single entry point: TLS, authentication/authorization, routing, rate-limiting, request audit logging.	–
Identity & Access Mgmt	Platform	Users, roles (QA_REVIEWER, QA_APPROVER, OWNER...), permissions, authentication, electronic signatures, central audit trail.	iam_db
Workflow Orchestration (Flowable)	Platform	Hosts the Flowable BPMN engine; owns all 11 process definitions, RuntimeService/TaskService, JavaDelegates, the unified Task Inbox, and cross-module orchestration.	workflow_db (ACT_*)
Notification Service	Platform	In-app and email notifications and escalations fired by workflow service tasks.	notification_db
File / Object Storage	Platform	Stores attachments, evidence and generated reports.	Object store (S3/MinIO)
Quality Events Service	Domain	Deviation, Nonconformance, Complaint lifecycles.	quality_events_db
CAPA Service	Domain	Corrective/preventive action lifecycle; consumed by all event/governance modules.	capa_db
Change & Risk Service	Domain	Change Control and Risk Management lifecycles.	change_risk_db
Audit & Supplier Service	Domain	Audit Management and Supplier Quality lifecycles.	audit_supplier_db
Document & Training Service	Domain	Document control and Training assignment/verification.	doc_training_db
Equipment Service	Domain	Equipment qualification, calibration, maintenance, re-qualification.	equipment_db
Event / Message Broker	Infra	Asynchronous domain events between services (Kafka or RabbitMQ).	–

Where Flowable fits in the split

In the current monolith the cross-module triggers are in-process `JavaDelegate` calls. In the target architecture the **Workflow Orchestration Service** becomes the single home of the Flowable engine and all BPMN process definitions, so the orchestration logic stays centralized while the business data is distributed:

- Each domain service, on a state change, calls the workflow service to `startProcessInstanceByKey()` or `completeTask()` — the same engine API as today, now over REST/gRPC.
- Flowable **service tasks** that previously created a record in another module now call the **target domain service's API** (or publish an event) instead of writing another module's tables directly. The cross-module trigger matrix is unchanged in meaning — only the transport changes.
- The **unified Task Inbox** is served by the workflow service, so users still see one inbox spanning all modules.
- Boundary timers and escalations continue to run inside the engine, calling the Notification Service.

Interaction patterns

Synchronous orchestration is used where an action must complete as part of a workflow step — e.g. a deviation disposition initiating a CAPA: the workflow service calls the CAPA service API and links the record.

Asynchronous events (over the broker) are used for fire-and-forget propagation and read-model updates — e.g. `DocEffective` → `Training`, or `SupplierScoreChanged`. Because business records now live in different services, multi-step cross-module flows are coordinated as **sagas** driven by the BPMN process, with compensating actions instead of a single database transaction.

Data ownership

Each service owns its database exclusively (**database-per-service**); no service reads another's tables. Cross-service references are stored as IDs (e.g. a CAPA holds `sourceType` + `sourceId` pointing at the originating deviation) and resolved via API or event, replacing the foreign keys used in the monolith. The shared regulatory concerns — audit trail and electronic signatures — are centralized in the IAM service so Part 11 evidence remains consistent across services.

Module-to-service mapping

QMS module	Target service	BPMN process (unchanged)
Deviation	Quality Events	deviationProcess
Nonconformance	Quality Events	ncProcess
Complaint	Quality Events	complaintProcess
CAPA	CAPA	capaProcess
Change Control	Change & Risk	changeControlProcess
Risk Management	Change & Risk	riskProcess
Audit	Audit & Supplier	auditProcess
Supplier Quality	Audit & Supplier	supplierProcess
Document Mgmt	Document & Training	documentProcess
Training	Document & Training	trainingProcess
Equipment	Equipment	equipmentProcess

Suggested migration path

A **strangler-fig** approach lets the split happen incrementally without a big-bang rewrite:

1. Stand up the **API Gateway** in front of the existing monolith; no behavior change.
2. Extract the **platform services first** — IAM (with e-signatures/audit), Notification, and the Workflow Orchestration Service (move the Flowable engine and BPMN definitions out of the monolith). The monolith then calls the workflow service instead of an embedded engine.
3. Carve out domain services one bounded context at a time, starting with the most independent (**Equipment**), then **Document & Training** and **Audit & Supplier**, and finally the highly-connected core (**Quality Events**, **CAPA**, **Change & Risk**). Split each service's tables into its own database as it is extracted.
4. Replace in-process cross-module delegate calls with service-API calls / events as each target service comes online.

This keeps the system releasable at every step and preserves the GxP audit trail throughout. If full microservices feel premature, the same six-context boundary can first be enforced as a **modular monolith** (one deployable, strict module packages) and split later with minimal refactoring.